

```
>>> import Image, stegpic
```

3. Open an image file in which you want to hide data:

```
>>> im=Image.open("lena.jpg")
```

4. Encode some text into the source image. This returns another Image instance, which you need to save to a new file:

```
>>> in2=stegpic.encode(in, 'This is the hidden text')
>>> in2.save('stegolena.jpg', 'JPEG')
```

5. Display both the images (with and without hidden data) to see if there is any visible change:

```
>>> in2=Image.open("stegolena.jpg")
>>> in2.show()
>>> in.show()
```

Use the `decode()` function to extract data from an image:

```
>>> in1=Image.open('stegolena.jpg')
>>> s=stegpic.decode(in1)
>>> data=s.decode()
>>> print data
This is the hidden text
```

Hiding encrypted data inside images

Instead of hiding plain-text data in images, let's encrypt that data (with the `ezPyCrypto` module) before hiding it. Since `Stegpic` accepts data in ASCII format, after encrypting data, we need to convert it to ASCII format.

1. Import modules into the interactive shell:

```
>>> import Image
>>> import stegpic
>>> import ezPyCrypto
```

2. Open an image in which you plan to hide data:

```
>>> im=Image.open("lena.jpg")
```

3. Create an encryption key and encrypt the message to ASCII format:

```
>>> k=ezPyCrypto.key(2048)
>>> enc=k.encryptToString('This is the hidden text')
```

4. Hide this encrypted message in the image, and save as a new file:

```
>>> in1=stegpic.encode(enc)
>>> in1.save('stegolena.jpg', 'JPEG')
```

The statements shown above are the bare bones of the procedure; we have omitted some commands that you will see in Figure 1, which displays the encrypted data, and the image files before and after hiding data.

Extracting and decrypting the hidden message

Note: Create the decryption key based on the same number you used when encrypting the data.

```
>>> import Image, stegpic, ezPyCrypto
>>> im=Image.open("stegolena.jpg")
>>> data=stegpic.decode(in)
>>> k=ezPyCrypto(2048)
>>> dec=k.decryptFromString(data)
>>> print dec
This is the hidden text
```

Here, we've used secret-key cryptography, where the same key is used by the sender and receiver. You could also use public-key cryptography with `ezPyCrypto`, but that is outside the scope of this article. 

References

- Jain Ankit, 'Steganography: A solution for data hiding'
- Robert Krenn, 'Steganography Implementation & Detection'
- Christian Cachin, 'Digital Steganography'
- James Madison, 'An Overview of Steganography'
- Neil F. Johnson, Sushil Jajodia, 'Exploring Steganography: Seeing the Unseen'
- Max Weiss, 'Principles of Steganography'
- T. Morkel, J.H.P. Eloff, M.S. Olivier, 'An Overview Of Image Steganography'
- Andreas Westfeld and Andreas Pfitzmann, 'Attacks on Steganographic Systems'
- <http://www.python.org>
- <http://domnll.org/stegpic/doc/>
- <http://www.freenet.org.nz/ezPyCrypto/>

By: Suhas Desai

The author works with Tech Mahindra Ltd. He writes on open source and security. In his free time, he volunteers for social causes. He can be reached at desai.suhas@gmail.com.

A Computer Magazine that does not talk about Music, Games, Photo albums—Only Business

Read Benefit: A Business Minded Computer Magazine



www.benefitmag.com